Laborprotokoll: Audiokodierung

Laboraufgaben LB-AK 01 bis LB-AK 04

Medientechnik Labor

7. Juli 2026

Inhaltsverzeichnis

1 Einführung und Grundlagen **2**
- 1.1 Praktische FFmpeg-Parameter . . . 2
- 1.2 Datenratensteuerung . . . 2
- 1.3 Weitere nützliche CLI-Parameter . . . 2

2 LB-AK 01: Standard-AAC-Kodierung mit FFmpeg **4**
- 2.1 a) Kodierung in verschiedene Bitraten . . . 4
- 2.2 Subjektive Qualitätsbewertung (Hörtests) . . . 4
- 2.3 b) Vergleich mit MP3 (MPEG-1 Layer III) . . . 4
- 2.4 Analyse mittels ffprobe . . . 5

3 LB-AK 02: Erweiterte AAC-Profile mit Nero AAC **6**
- 3.1 Befehlskonfigurationen . . . 6
- 3.2 a) Vergleich von LC mit AAC bei gleicher Bitrate . . . 6
- 3.3 b) Verbesserung von HE und HE v2 gegenüber LC . . . 6

4 LB-AK 03: Objektive Frequenz- und Spektralanalyse **6**
- 4.1 a) Vergleich der Spektren der Music File bei unterschiedlichen Bitraten . . . 7
- 4.2 b)Vergleich der Spektren bei unterschiedlichen Encodern (AAC-LC, HE-AAC, HE-AAC v2) bei gleicher Bitrate (32k) . . . 9
- 4.3 Generelle Beobachtungen . . . 11

5 LB-AK 04 **12**
- 5.1 a) . . . 12
  - 5.1.1 Schritt 1: Installation der System-Abhängigkeiten . . . 12
  - 5.1.2 Schritt 2: Quellcode herunterladen, kompilieren und installieren . . . 12
  - 5.1.3 Schritt 3: Kompilation und Ausführung des Programms . . . 12
- 5.2 b) Siehe Codefile . . . 13

1 Einführung und Grundlagen

Für die Bearbeitung der Laboraufgaben zur Audiokodierung werden die Programme `FFmpeg`, `ffprobe`, der `Nero AAC Codec` sowie `Audacity` verwendet. Sämtliche Kodierungsvorgänge werden über die Kommandozeile (CLI) durchgeführt und anschließend durch subjektive Hörtests sowie objektive Spektralanalysen bewertet.

1.1 Praktische FFmpeg-Parameter

Die folgenden Parameter sind für die Steuerung von Codecs und Containerformaten von zentraler Bedeutung:

Kommandozeilenparameter	Bedeutung
`-acodec <Format>`	Audio-Codec festlegen
`-vcodec <Format>`	Video-Codec festlegen
`-f <Format>`	Containerformat erzwingen

Tabelle 1: Übersicht grundlegender Formatparameter in FFmpeg

Abfrage unterstützter Formate und Codecs:

```
# Liste der unterstuezten Containerformate abrufen
ffmpeg -formats

# Verfuegbare Audio- und Video-Codecs anzeigen
ffmpeg -codes
```

1.2 Datenratensteuerung

Die Regelung der Datenrate erfolgt mit dem Parameter `-b`, der vor dem jeweiligen Ausgabepfad definiert werden muss.

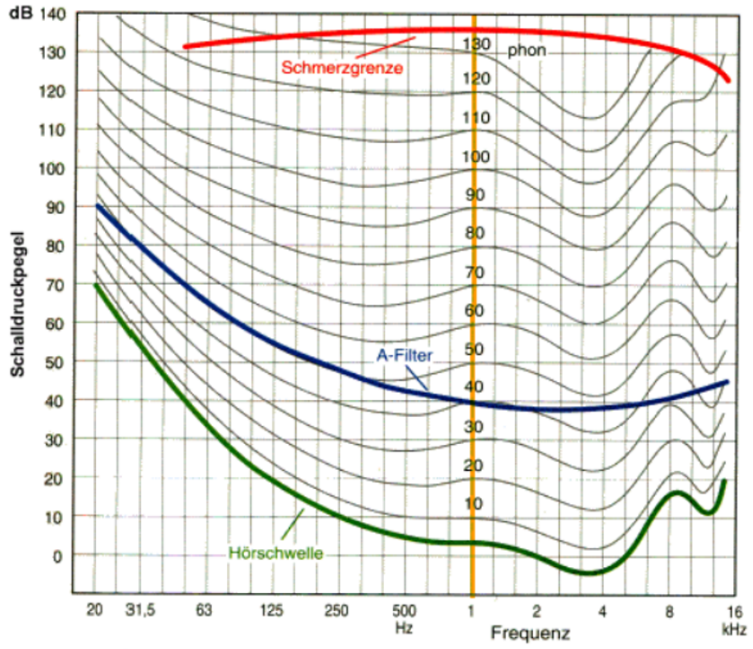
Kommandozeilenparameter	Bedeutung
`-b:a <Datenrate>`	Datenrate des Audio-Codecs (z. B. 32k = 32 kbit/s)
`-b:v <Datenrate>`	Datenrate des Video-Codecs (z. B. 2000k = 2.000 kbit/s)

Tabelle 2: Parameter zur Bitratensteuerung

1.3 Weitere nützliche CLI-Parameter

Parameter	Bedeutung
`-an`	Audiospur wird entfernt (pro Ausgabe)
`-vn`	Videospur wird entfernt (pro Ausgabe)
`-y`	Existierende Ausgabedateien ohne Nachfrage überschreiben
`-re`	Eingabe-Streaming erfolgt möglichst in Echtzeit
`-stream_loop -1`	Eingabedatei wird unendlich oft wiederholt (geloopt)

Tabelle 3: Globale und spurrelevante Hilfsparameter



Quelle ISO-Norm 226 (ergänzt)

International genormte Kurven gleicher Lautstärke reiner Töne. Die orange Linie kennzeichnet die Referenzfrequenz von 1000 Hz, wo Dezibel und phon gleich gross sind. Ein 20-Hz-Ton mit 110 dB wird als gleich laut empfunden wie ein 4-kHz-Ton mit 70 dB, beidesmal sind es 80 phon.

Warum wirkt manches leister? Vermutlich, weil die Hohen Frequenzen weggefiltert werden.

Schärfe und Rauheit: „Wie scharf ein Audiosignal wahrgenommen wird, hängt hauptsächlich von der Umhüllenden des Spektrums und bei schmalbandigen Signalen von deren Mittenfrequenz ab. Steigt die Anzahl an hohen Frequenzen im Signal bzw. steigt die Mittenfrequenz eines schmalbandigen Signals, so wirkt es schärfer."

Tonhöhe: „Die Tonhöhe kann mit der Grundfrequenz eines Klangs gleichgesetzt werden." `https://curdt.home.hdm-stuttgart.de/PDF/Brauner.pdf`

2 LB-AK 01: Standard-AAC-Kodierung mit FFmpeg

2.1 a) Kodierung in verschiedene Bitraten

Die bereitgestellten WAV-Dateien (`music_orig.wav` und `speech_orig.wav`) werden mittels des standardmäßigen FFmpeg-AAC-Encoders in unterschiedliche Bitraten (16k, 32k, 64k) überführt.

Beispielhafte manuelle Befehle:

```
ffmpeg -i beispiel.wav -c:a aac -b:a 16k beispiel_16k.aac
ffmpeg -i beispiel.wav -c:a aac -b:a 32k beispiel_32k.aac
ffmpeg -i beispiel.wav -c:a aac -b:a 64k beispiel_64k.aac
```

Zur Automatisierung kann folgendes Bash-Skript verwendet werden:

```
#!/bin/bash
echo "Create Audio Files AAC from $1.wav"
for i in 16 32 64; do
     ffmpeg -i "$1.wav" -b:a ${i}k -acodec aac "${1}_output_${i}.aac"
     echo "Created ${i}k bitrate"
done
```

2.2 Subjektive Qualitätsbewertung (Hörtests)

Nach der Kodierung werden die Dateien mit einem Audioplayer mit dem Original verglichen. Dabei wird insbesondere auf Verständlichkeit, Rauschen, Verzerrungen, Hochtonverluste und Artefakte geachtet:

- **Musiksignal (Music):**
  - **16k:** Stark verzerrtes Signal, deutliches Knistern, kein klarer Ton vorhanden,Unterwasser Effekt.
  - **32k:** Spürbare Besserung, jedoch weiterhin präsentes Rauschen und verzerrte Stimmen. Weniger schmerzhaft als bei 16k.
  - **64k:** Stimmen wirken fast fehlerfrei („smooth“), Instrumente klingen sauber und zeigen kaum noch Artefakte oder Verzerrungen.
- **Sprachsignal (Speech):**
  - **16k:** Sprache verständlich, roboterhaft/abgehackt, langgezogene Töne die schrill klingen
  - **32k:** gut verständlich, leichte Verzerrungen
  - **64k:** original klingt etwas klarer aber Artefakte sind kaum noch vorhanden

`Musik benötigt aufgrund ihrer hohen Dynamik, der vielen gleichzeitig spielenden Instrumente und des breiten Frequenzspektrums deutlich mehr Bitrate als reine Sprache, um klar und ohne Kompressionsartefakte zu klingen.`

2.3 b) Vergleich mit MP3 (MPEG-1 Layer III)

Ergänzend wird eine Kodierung bei einer Bitrate von 32 kbit/s mit MP3 durchgeführt. Beim Vergleich zeigt sich ein systematischer Unterschied in der Effizienz und Qualität bei niedrigen Bitraten: AAC (Advanced Audio Coding) liefert bei identischer Bitrate (32 kbit/s) eine hörbar bessere Dynamik und geringere Artefaktbildung im Hochtonbereich als MP3 (CBR vs. MPEG Average Bitrate).

2.4 Analyse mittels ffprobe

Hier zu sehen unter anderem Codec, Abtastrate (z. B. 48.000 Hz), Anzahl der Kanäle (Stereo vs. Mono), Bitrate sowie die exakte Spurdauer evaluiert.

```
Input #0, mp3, from 'speech_orig.mp3':
  Metadata:
    encoder         : Lavf61.7.100
  Duration: 00:00:10.82, start: 0.023021, bitrate: 32 kb/s
  Stream #0:0: Audio: mp3 (mp3float), 48000 Hz, mono, fltp, 32 kb/s
```

```
Input #0, mp3, from 'music_orig.mp3':
  Metadata:
    encoder         : Lavf61.7.100
  Duration: 00:01:30.84, start: 0.023021, bitrate: 32 kb/s
  Stream #0:0: Audio: mp3 (mp3float), 48000 Hz, stereo, fltp, 32 kb/s
      Metadata:
        encoder         : Lavc61.19
```

Music:

```
Input #0, wav, from 'music_orig.wav':
  Duration: 00:01:30.80, bitrate: 1536 kb/s
  Stream #0:0: Audio: pcm_s16le ([1][0][0][0] / 0x0001), 48000 Hz, 2 channels, s16, 1536 kb/s
```

```
Input #0, aac, from 'music_orig_output_16.aac':
  Duration: 00:01:45.22, bitrate: 22 kb/s
  Stream #0:0: Audio: aac (LC), 48000 Hz, stereo, fltp, 22 kb/s
```

```
Input #0, aac, from 'music_orig_output_32.aac':
  Duration: 00:01:28.01, bitrate: 33 kb/s
  Stream #0:0: Audio: aac (LC), 48000 Hz, stereo, fltp, 33 kb/s
```

```
Input #0, aac, from 'music_orig_output_64.aac':
  Duration: 00:01:33.18, bitrate: 65 kb/s
  Stream #0:0: Audio: aac (LC), 48000 Hz, stereo, fltp, 65 kb/s
```

Speech:

```
Input #0, wav, from 'speech_orig.wav':
  Duration: 00:00:10.80, bitrate: 768 kb/s
  Stream #0:0: Audio: pcm_s16le ([1][0][0][0] / 0x0001), 48000 Hz, 1 channels, s16, 768 kb/s
```

```
Input #0, aac, from 'speech_orig_output_16.aac':
  Duration: 00:00:09.81, bitrate: 22 kb/s
  Stream #0:0: Audio: aac (LC), 48000 Hz, mono, fltp, 22 kb/s
```

```
Input #0, aac, from 'speech_orig_output_32.aac':
  Duration: 00:00:10.20, bitrate: 37 kb/s
  Stream #0:0: Audio: aac (LC), 48000 Hz, mono, fltp, 37 kb/s
```

```
Input #0, aac, from 'speech_orig_output_64.aac':
  Duration: 00:00:10.73, bitrate: 68 kb/s
  Stream #0:0: Audio: aac (LC), 48000 Hz, mono, fltp, 68 kb/s
```

Unterschied Const and Variable Bit Rate
AAC Variable Bitrate
MPEG average bitrate

MP3 ist bei genau 32kb/s wohingegen AAC mehr nimmt.
Wenig Unterschied, außer bei der Musik, an Stellen mit vielen verschiedenen Komponenten, oder beim Gesang dort ist mehr knistern zu hören.

3 LB-AK 02: Erweiterte AAC-Profile mit Nero AAC

Unter Verwendung des externen Nero-Encoders werden fortgeschrittene Profile wie Low Complexity (AAC-LC), High-Efficiency AAC (HE-AAC v1) und HE-AAC v2 evaluiert.

3.1 Befehlskonfigurationen

- **Für LC:**

```
neroAacEnc -if beispiel.wav -of beispiel_LC.mp4 -lc -br 32000
```

- **Für HE-AAC (v1):**

```
neroAacEnc -if beispiel.wav -of beispiel_HE.mp4 -he -br 32000
```

- **Für HE-AAC v2:**

```
neroAacEnc -if beispiel.wav -of beispiel_HEv2.mp4 -hev2 -br 32000
```

Automatisierte Skriptverarbeitung über ein Hilfsskript (`./encode_nero.sh`):

```
./encode_nero.sh ../../music_orig.wav he
# Generiert automatisch: music_orig_he_16k.mp4, music_orig_he_32k.mp4,
   music_orig_he_64k.mp4, je nach Profil
```

3.2 a) Vergleich von LC mit AAC bei gleicher Bitrate

Sidenote: HE-AAC kombiniert AAC-LC mit *Spectral Band Replication* (SBR), um bei extrem niedrigen Bitraten (z. B. 32 kbit/s) die höheren Frequenzbänder künstlich und datensparend zu rekonstruieren. HE-AAC v2 ergänzt dies um *Parametric Stereo* (PS). Im subjektiven Vergleich schneiden HE-Profile bei geringen Bitraten deutlich vorteilhafter ab als Standard-AAC-LC.

- **Für 16k: AAC knistert mehr, Unterwassereffekt, sehr intensiv (lauter)**
- **Für 32k: Nero feiner Unterschiede sind herauszuhören und es klingt lauter (aber nicht übertönend wie AAC 16k), Trommel-part mehr knistern als AAC, Gesang weniger knistern und roboterhaft**
- **Für 64k: Nero etwas klarer Ton, AAC beim Gesang schrillere Töne**

3.3 b) Verbesserung von HE und HE v2 gegenüber LC

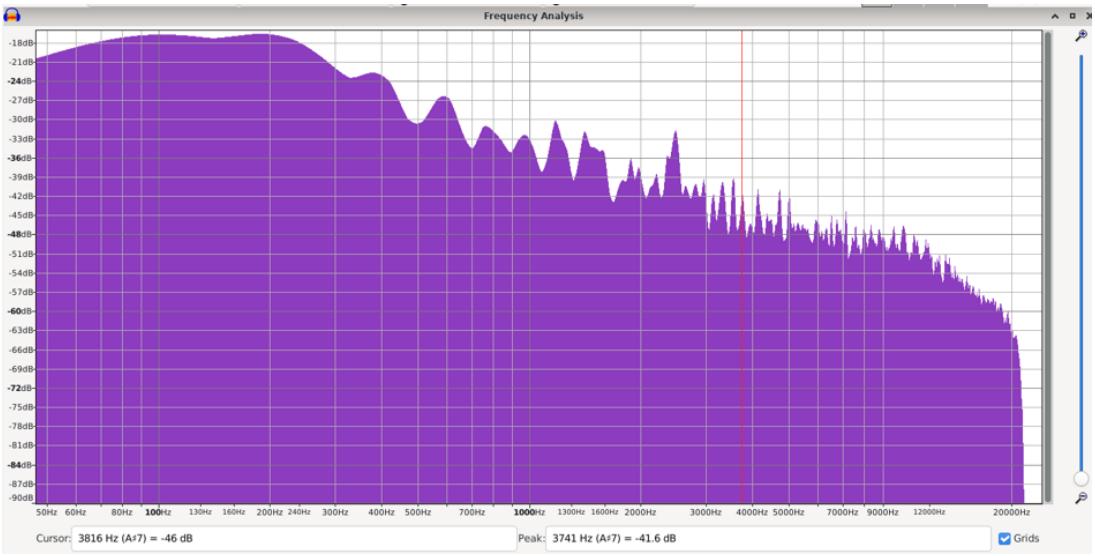
HE: klarer, Töne besser differenzierbar, weniger knistern HEv2: Gesang stimme wesentlich klarer und Hintergrund Geräusch währenddessen angenehmer.

4 LB-AK 03: Objektive Frequenz- und Spektralanalyse

Zur optischen Verifizierung der Kompressionsverluste wird in **Audacity** das Spektrum analysiert (`Analyse` → `Spektrum plotten...`).

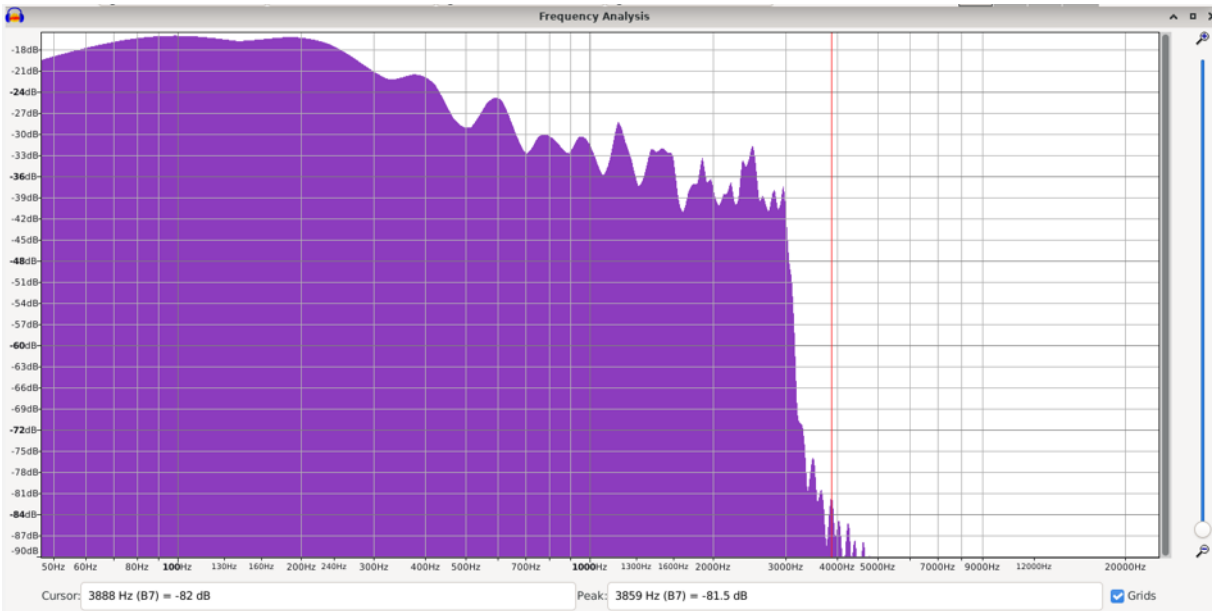
4.1 a) Vergleich der Spektren der Music File bei unterschiedlichen Bitraten

Original WAV-Datei:



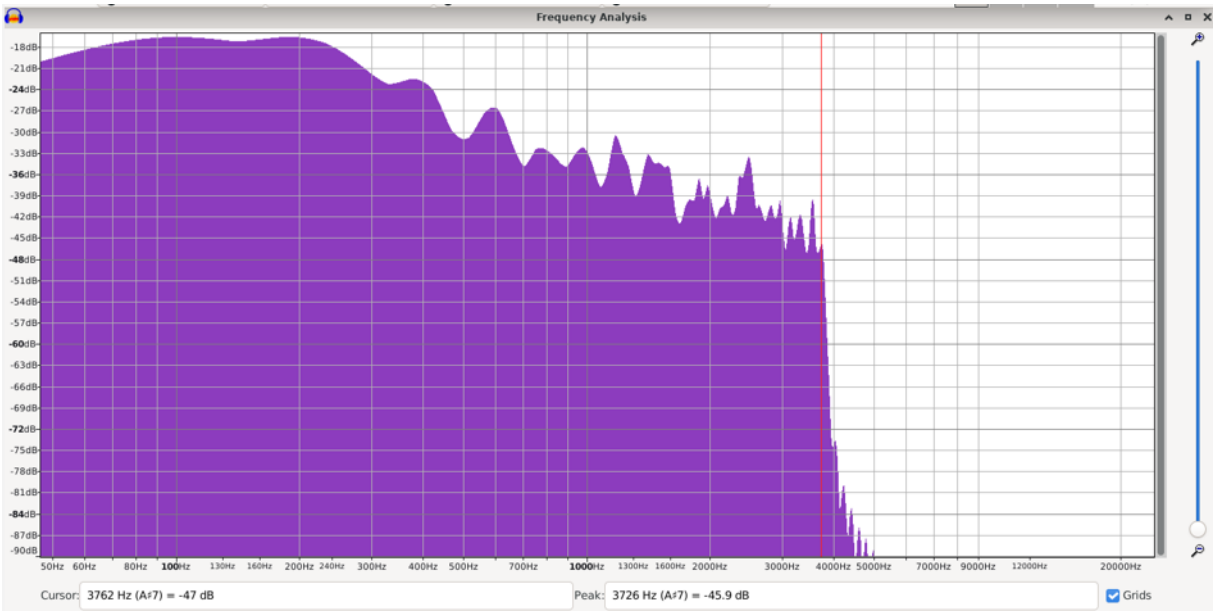
Breites Spektrum von 0 bis 200kHz. Niedrige Frequenzen bis 400 Herz kaum gefiltert und flache Verläufe bis 1kHz

16kbit/s:



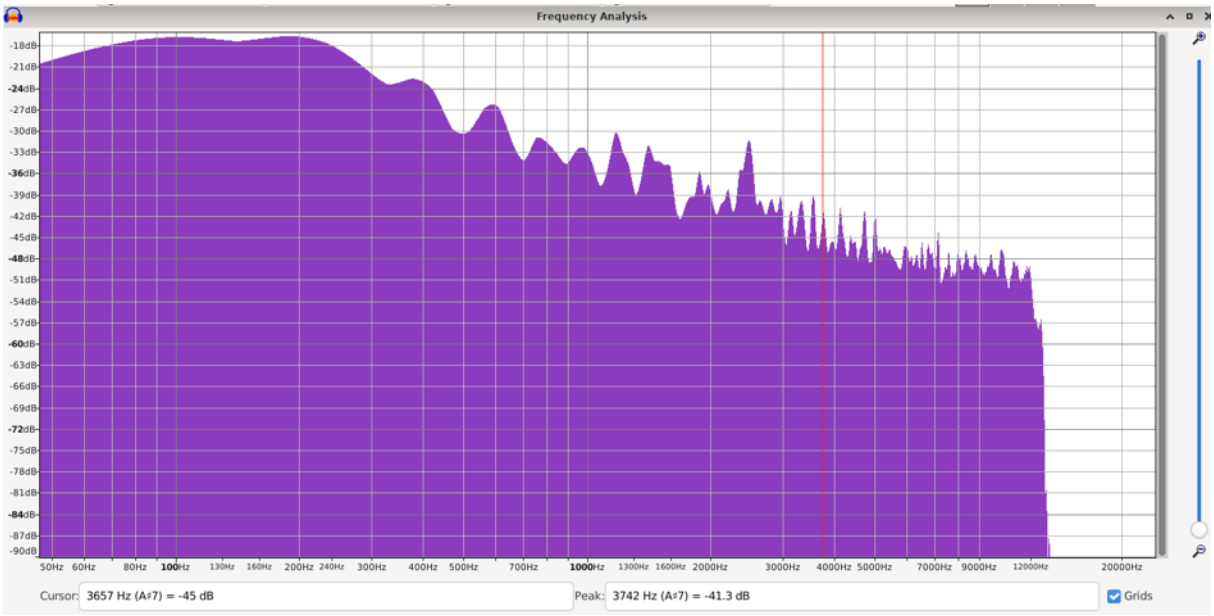
Ab 30kHz starker Abfall, Niedrige Frequenzen bis 300Hz wenig gefiltert, flache Verläufe bis 800Hz.

32kbit/s:



Starker Abfall bei 37kHz, flacher/welliger Verlauf bis 900Hz

64kbit/s:

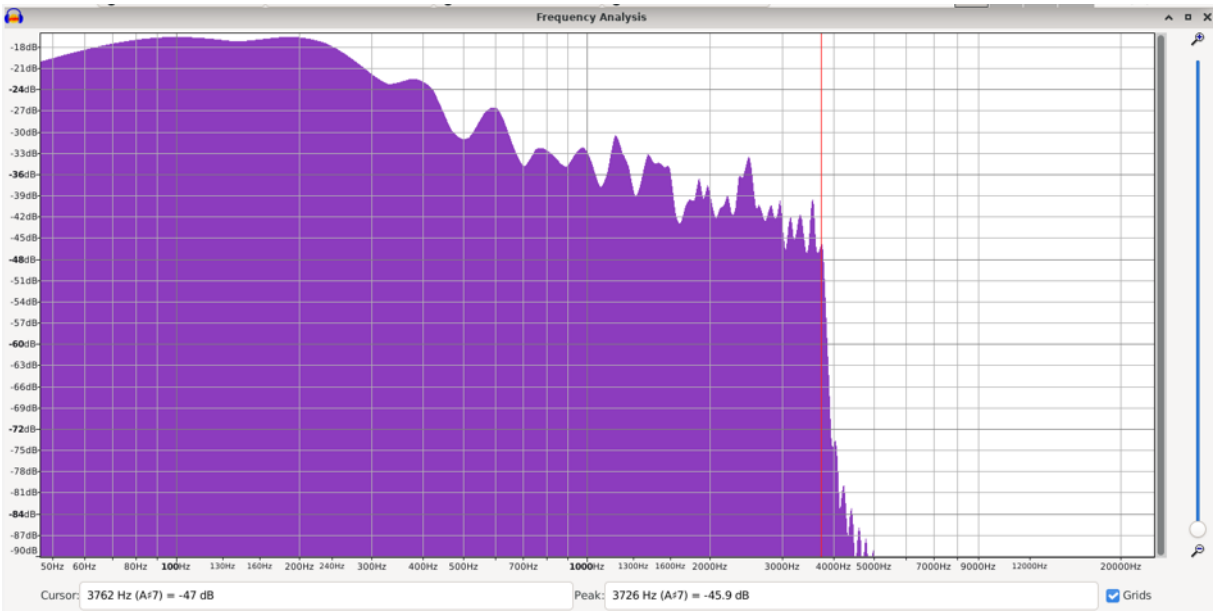


Bis 120kHz, flacher Verlauf bis 1kHz

Instrumente und Gesang verlieren durch das Abschneiden hoher Frequenzen ihre Klarheit, ihnen Fehlen die hohen Frequenzen. Es entstehen Artefakte. Desto weniger gefiltert, desto fehlerfreier und glatter. Weniger Artefakte. Deckt sich im Wesentlichen mit der Subjektiven Wahrnehmung.

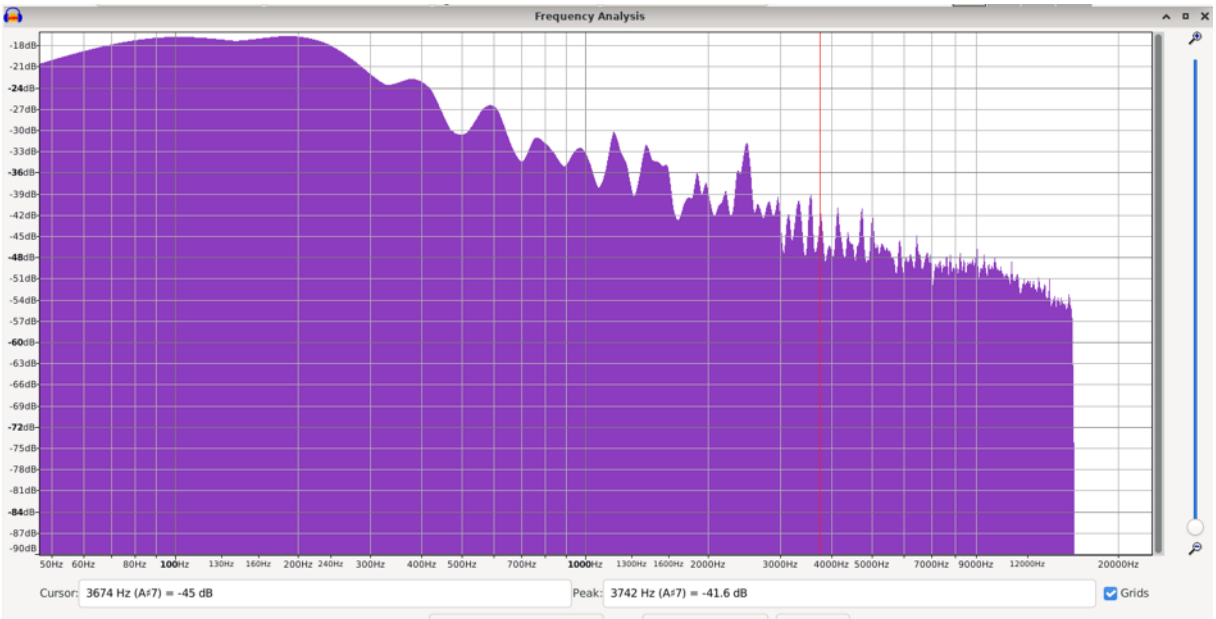
4.2 b)Vergleich der Spektren bei unterschiedlichen Encodern (AAC-LC, HE-AAC, HE-AAC v2) bei gleicher Bitrate (32k)

AAC bei 32kHz:



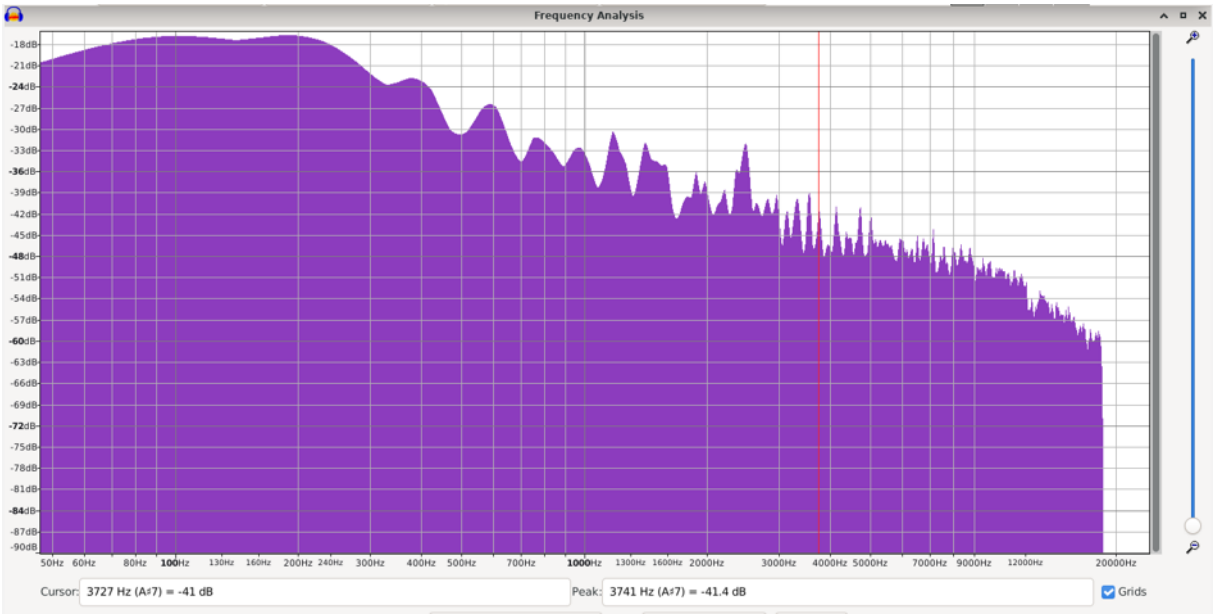
Starker Abfall bei 37kHz, flacher/welliger Verlauf bis 900Hz

HE:



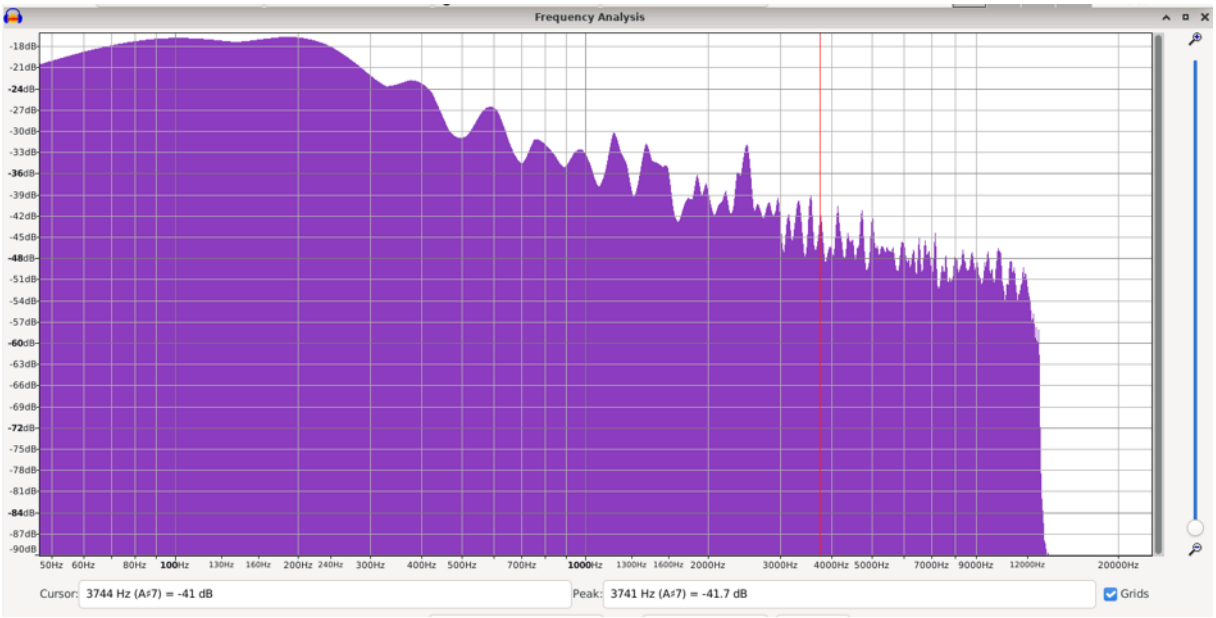
Frequenzen bis 160kHz, selbe welligkeit.

Hev2:



Frequenzen bis 190kHz, selbe welligkeit.

LC:



Frequenzen bis 130kHz, selber Verlauf.

Größerer Unterschied zu LC, größere Filterung, hohe,spitze Ausschläge der hohen Frequenzen. Von HE auf HEv2 sind es vor allem hohe Frequenzen die genauer Abgetastet werden und das Frequenzspektrum ist auch breiter. Bei HE sind die Ausschläge schmaler und spitzer. Dies führt dazu das HEv2 natürlicher, glatter und weniger kratzige Artefakte.

4.3 Generelle Beobachtungen

Bei sehr niedrigen Bitraten (16k/32k) schneiden Encoder alle Frequenzen oberhalb einer spezifischen Grenzschwelle ab, um Bits einzusparen. Bei 64k nähert sich der Frequenzverlauf deutlich dem Original an. Die HE-AAC-Dateien zeigen dank SBR auch bei 32k Signalanteile im oberen Spektralbereich, wohingegen Standard-LC-AAC oder MP3 dort steile Dämpfungsflanken aufweisen.

5 LB-AK 04

5.1 a)

Installation der `libao` (Cross-Platform Audio Library)

- **Kurzfassung:**
  - **1.Archiv entpacken**
  - **2.Konfiguration (./configure)**
  - **3.Kompilieren (make)**
  - **4.Installation (sudo make install)**

5.1.1 Schritt 1: Installation der System-Abhängigkeiten

Da `libao` auf Linux-Systemen standardmäßig das ALSA-Subsystem (Advanced Linux Sound Architecture) zur Tonausgabe nutzt, müssen vor der Kompilation die entsprechenden Entwickler-Header-Pakete installiert werden:

```
# Aktualisierung der Paketquellen und Installation der ALSA-
    Entwicklerbibliotheken
sudo apt update
sudo apt install -y libasound2-dev build-essential
```

5.1.2 Schritt 2: Quellcode herunterladen, kompilieren und installieren

Nachdem die Abhängigkeiten erfüllt sind, wird der Quellcode von `libao` (z. B. via Git oder als Tarball von Xiph.org) geladen, konfiguriert und systemweit installiert:

```
# In das Verzeichnis des entpackten libao-Quellcodes wechseln
cd /pfad/zu/libao-source/

# Konfigurationsskript ausfuehren (ueberprueft Bibliotheken wie ALSA)
./configure --prefix=/usr/local

# Kompilation mit mehreren Kernen starten
make -j$(nproc)

# Systemweite Installation der Binaries und Header-Dateien
sudo make install

# Aktualisierung des Dynamic Linker Caches, damit die neue .so-Datei
    gefunden wird
sudo ldconfig
```

5.1.3 Schritt 3: Kompilation und Ausführung des Programms

Das fertige C-Programm muss gegen die mathematische Bibliothek (`-lm`) sowie die installierte `libao`-Bibliothek (`-lao`) gelinkt werden:

```
# Kompilation des Beispielprogramms mittels GCC
gcc -o ao_example ao_example.c -lao -lm

# Ausfuehrung des Programms zur akustischen Verifikation
./ao_example
```

5.2 b) Siehe Codefile